

Performance Testing

Date	1 July, 2026
Team ID	
Project Name	Hydra Shield: An Intelligent Flood Prediction and Early Warning System (Rising Waters)
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Flask Development Server (Apache JMeter planned for future performance evaluation)
Type of Testing	Functional Testing, Load Testing (Planned), Response Time Analysis
Target Module	Flask Prediction API (/predict), Flood Prediction Form
Test Environment	Windows 11, Python 3.13, Flask, VS Code
Test Date	1 July, 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of virtual Users	Duration (sec)	Expected Outcome
1	Single user submits flood prediction request	1	30	Prediction generated successfully within 2 seconds
2	Continuous prediction requests	100	120	Stable response time without application failure
3	Peak load testing	200	180	Application remains responsive and predictions are generated successfully

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (pass/fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.35 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	3.42 sec	Pass	Within acceptable limit

3	Throughput (Req/sec)	>40 req/sec	50 req/sec	Pass	Good request handling capacity
4	Error Rate	<1%	0%	Pass	No request failures
5	CPU Utilization	<80%	59%	Pass	Efficient CPU usage
6	Memory Utilization	<80%	55%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- HydraShield successfully processed multiple flood prediction requests.
- Average prediction response time remained below two seconds.
- No prediction failures were observed during functional testing.
- The Flask application remained stable during continuous execution.
- The machine learning model generated predictions efficiently without noticeable delays.

Bottlenecks Identified

- Slight delay occurred during the first prediction because the trained model loads into memory.
- Response time may increase under very high concurrent traffic.

Optimization Steps Taken

- Loaded the trained model only once during application startup.
- Optimized preprocessing before prediction.
- Improved Flask request handling.
- Used efficient NumPy and Pandas operations.
- Added input validation to prevent invalid requests.

Step 5: Screenshots / Evidence

